

```
Tactical Shadow RAT #
```

```
## דוח פרויקט מסכם: פיתוח מערכת שליטה מרחוק (RAT)
```

```
**קורס:** פיתוח התקפי (Offensive Python)
```

```
**מגיש:** heikel muhammad
```

```
**שם הכלי:** Tactical Shadow RAT
```

```
---
```

```
## 1. מבוא וסקירת המערכת
```

במסגרת פרויקט הגמר פותח כלי שליטה מרחוק (Remote Access Trojan) מוסווה, המשלב יכולות איסוף מודיעין, שליטה על מערכת הקבצים והסוואת מטען ייעודי.

המערכת נבנתה בארכיטקטורת Client-Server על גבי פרוטוקול TCP, עם דגש על יציבות תקשורתית, חוסן במקרה של ניתוקים ועקיפת מנגנונים בסיסיים של הגנה.

```
---
```

```
## 2. פירוט טכני ועמידה בדרישות המטלה
```

המערכת נבנתה בהתאם לדרישות הקורס וכוללת את היבטי הפיתוח הבאים:

- ****GUI צד התוקף:**** הממשק נבנה ב-Tkinter וכולל חלון לוגים דינמי המציג פלט מהקורבן בזמן אמת, כפתורי שליטה מהירים ושורת פקודה אינטראקטיבית.
- ****פרוטוקול תקשורת TCP:**** התקשורת מבוססת על Sockets ב-TCP כדי להבטיח העברת נתונים אמינה ולמנוע איבוד מידע בעת שליחת קבצים כבדים כמו צילומי מסך.

- ****הצפנת תעבורה (Obfuscation):**** כל המידע עובר כ-JSON מוצפן באמצעות אלגוריתם XOR סימטרי עם מפתח קבוע, מה שמקשה על מנגנוני ניטור רשת לזהות חתימות תקשורת סטנדרטיות.

- ****חוסן ויציבות:**** צד הנתקף מבצע Auto-Reconnect אוטומטי במקרה של ניתוק, וכך נמנע קריסת הכלי בעת נפילת רשת או ריסטארט.

3. כלים התקפיים שהוטמעו (Capabilities)

המערכת כוללת כלים מתקדמים לשליטה ולריגול:

- ****Keylogger:**** הקלטת הקשות מקלדת ושמירתן ב-`logs.txt`.

- ****Remote Shell:**** הפעלת פקודות מערכת (CMD) בצד הקורבן וקבלת פלט מלא ישירות לממשק התוקף.

- ****Screenshot:**** לכידת צילום מסך של הקורבן, קידוד Base64 ושליחתו להצגה ב-GUI.

4. הכלי החדש (למידה עצמית): (File Hashing (SHA-256

כחלק מלמידה עצמית הוטמע מנגנון גיבוב SHA-256.

- ****הצורך:**** שמות קבצים ניתנים לשינוי בקלות, ולכן הזיהוי צריך להתבסס על תוכן.

- ****היישום:**** חישוב טביעת אצבע דיגיטלית ייחודית לכל קובץ.

- ****שימושים:****

- זיהוי קבצים רגישים לפי תוכן, גם אם שם הקובץ שונה.
- אימות שלמות קבצים, למשל בדיקת קובץ הלוג `logs.txt` כדי לוודא שלא שונה.

5. אריזה כסוס טרויאני (Trojan Implementation)

הלקוח נבנה כסוס טרויאני ומוסווה באמצעות תמונת ``image.jpg``:

- קובץ הלקוח מאוחד עם תמונת סוואה בפורמט JPEG.
- בהפעלה, המשתמש רואה את התמונה נפתחת במציג התמונות של Windows.
- הקוד האמיתי של ה-RAT מתחיל לרוץ ברקע כ-Background Process ללא חלון גלוי.

6. רכיבי הפרויקט

- ``server.py`` – צד התוקף עם ממשק GUI, שליחת פקודות וקבלת פלט.
- ``client.py`` – צד הנתקף (סוס טרויאני) שמתחבר לשרת, מקבל פקודות ומבצע אותן.
- ``README.md`` – דוח פרויקט והנחיות שימוש.

7. התקנה והפעלה

דרישות

- Python 3.8 או גרסה חדשה יותר
- ספריות נוספות:
 - ``pynput``
 - ``Pillow``

התקנה

bash``

`pip install pynput pillow`

הפעלה

1. עדכן ב-`client.py` את כתובת השרת ופורט:
`"SERVER\_HOST = "192.168.1.140` -
`SERVER\_PORT = 8080` -
2. הרץ את השרת:

```
bash``  
python server.py  
``
```

3. הרץ את הלקוח במכונת הנתקף:

```
bash``  
python client.py  
``
```

4. השתמש ב-GUI לשליטה:

- `Screenshot` - צילום מסך וקבלת תמונה ב-GUI
- `Keylogger Logs` - שליחת לוג מקלדת
- `(Get Hash (logs.txt` - חישוב SHA-256 של קובץ
- `TERMINATE` - ניתוק וסגירת הלקוח
- שורת פקודה חופשית - שליחת פקודת shell מרחוק

8. מסקנה

הפרויקט מדגים שילוב של הנדסה חברתית, קריפטוגרפיה ותכנות התקפי מתקדם. המערכת עומדת בדרישות הקורס ומספקת פתרון שליטה חזק, מוצפן ובלתי נראה לצד הנתקף.